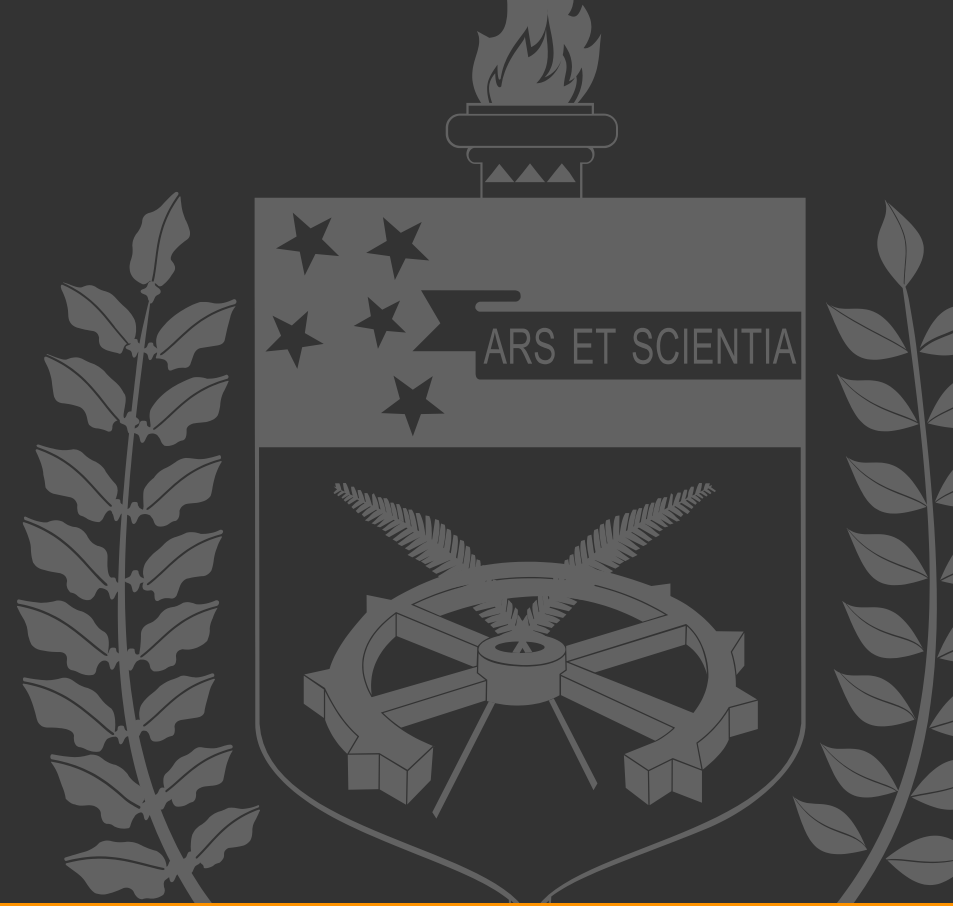


Programação Orientada a Objetos

Herança, Associações e Polimorfismo



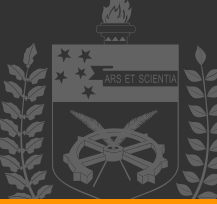
Universidade Federal de Santa Catarina – UFSC

Centro Tecnológico de Joinville – CTJ

Prof. Arthur Gabardo

2026.1

Relembrando

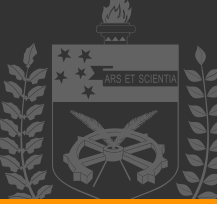


- Classes
 - Abstração/forma dos objetos
- Objetos
 - Instanciações de classes
- Atributos → características
 - Públicos, privados e ...
 - Protegidos (mais na aula de hoje)
- Métodos → ações
 - Garantem encapsulamento e consistência

```
#include <string>

class Pessoa {
private:
    std::string nome;
    static unsigned int contPessoas;

public:
    Pessoa();
    ~Pessoa();
    void apresentar() const;
    static unsigned int getTotalPessoas();
};
```



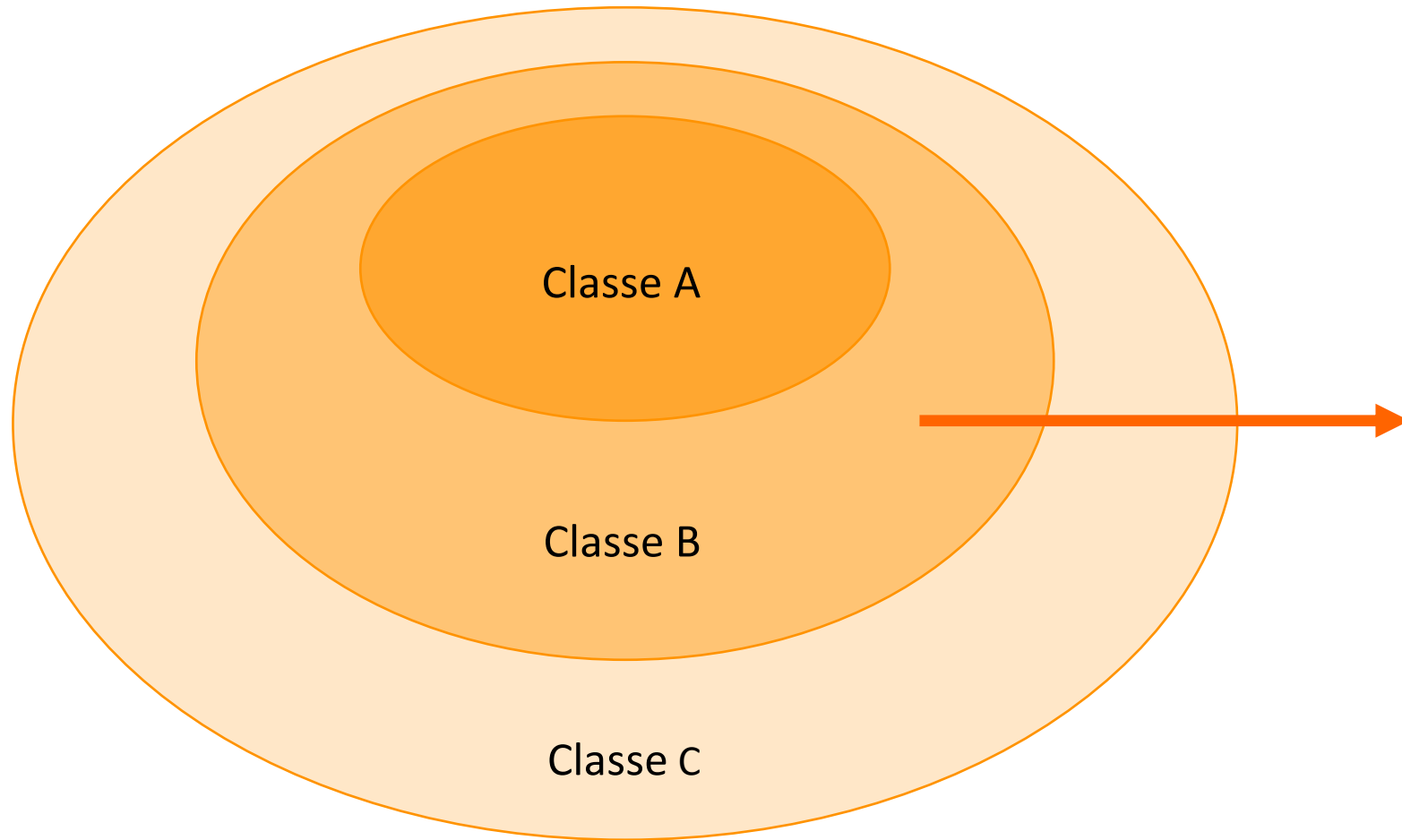
- Relacionamentos entre classes
 - Herança
 - Lei de Substituição de Liskov
 - Associação
 - Composição
 - Agregação
- Polimorfismo
 - Estático
 - Dinâmico



- Como representar diferentes entidades que compartilham comportamentos e características?
 - Compartilhando similaridades
 - Preservando as diferenças
- Facilita a manutenção do sistema
 - Maior legibilidade do código
 - Não necessita de refatoração completa
 - Alteração em poucas partes
- Reutilização de código
 - Reutilizar não é copiar e colar o mesmo bloco em vários lugares



- A herança permite que uma classe herde membros de outra
 - Base/superclasse → classe que cede seus membros
 - Derivada/subclasse → classe que herda os membros
- Reflete relações do tipo "*é um(a)*":
 - Um aluno é uma pessoa
 - Um cachorro é um animal
 - ...
- Promove especialização
 - Estende os membros da classe base



Classe B é:

- Subclasse de A
- Superclasse de C

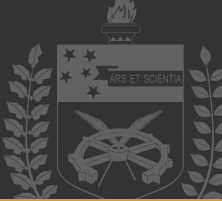
ou

- Derivada de A
- Base de C

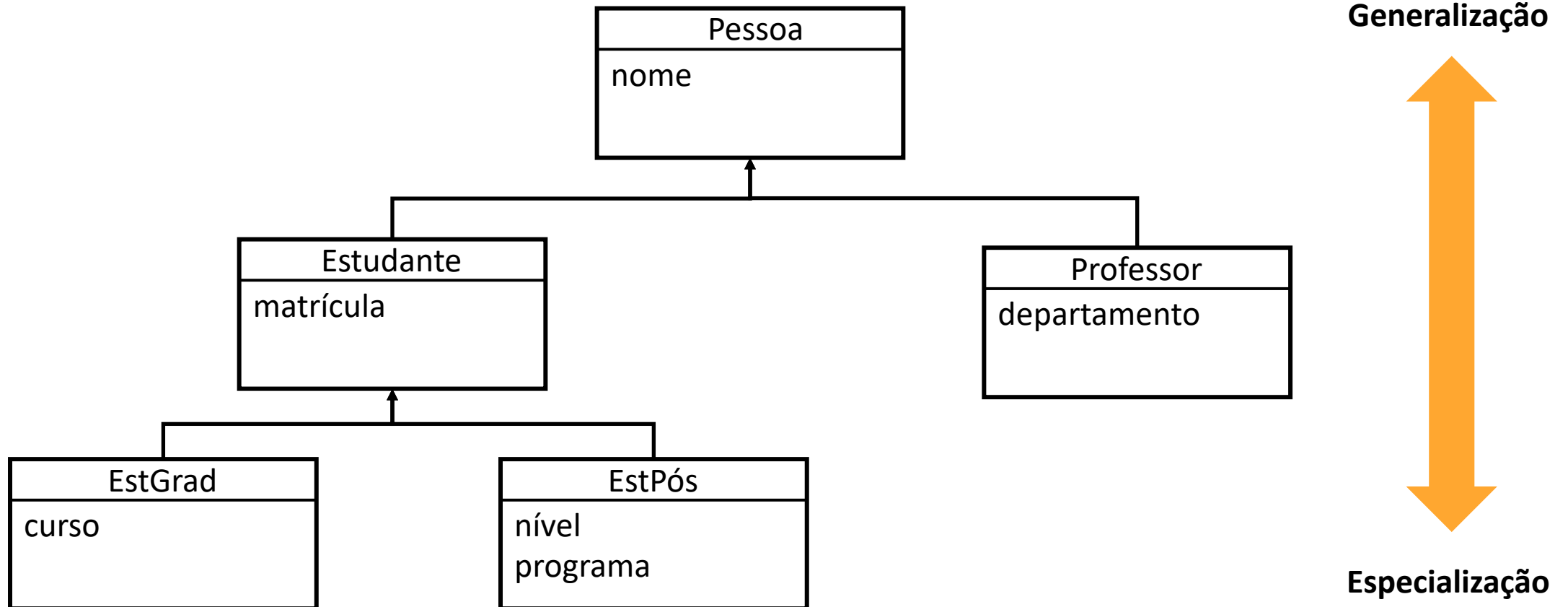


- A superclasse/base sempre tem seus atributos/métodos em comum com suas subclasses/derivadas
- Diz-se que as subclasses estendem a classe
 - Promovem a especialização das características e comportamentos das classes
- Diferentes subclasses podem herdar as características de uma mesma superclasse

Herança



- O conjunto de classes relacionadas por herança constitui uma hierarquia de classes
 - Árvore de heranças



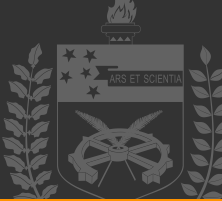
Implementação



```
class Pessoa {  
    string nome;  
};  
  
class Estudante : public Pessoa {  
    int matricula;  
};  
  
class Professor : public Pessoa {  
    string departamento;  
};  
  
class EstGrad : public Estudante {  
    string curso;  
};
```

- Possível herança indireta
 - Estudante herda Pessoa
 - EstGrad herda Pessoa de Estudante
- Possível ser superclasse de diferentes subclasses
 - Estudante herda Pessoa, e
 - Professor herda Pessoa
- Atributos privados não são diretamente acessíveis

Modificadores de acesso



```
class Pessoa {  
    protected:  
        string nome;  
};  
  
class Estudante : public Pessoa {  
    public:  
        int matricula;  
};  
  
class EstGrad : private Estudante {  
    string curso;  
};
```

Acesso na classe derivada:

public: acessível de qualquer lugar

protected: acessível na classe e nas derivadas

private: só acessível dentro da própria classe

- Membros privados da classe base não são acessíveis na classe derivada

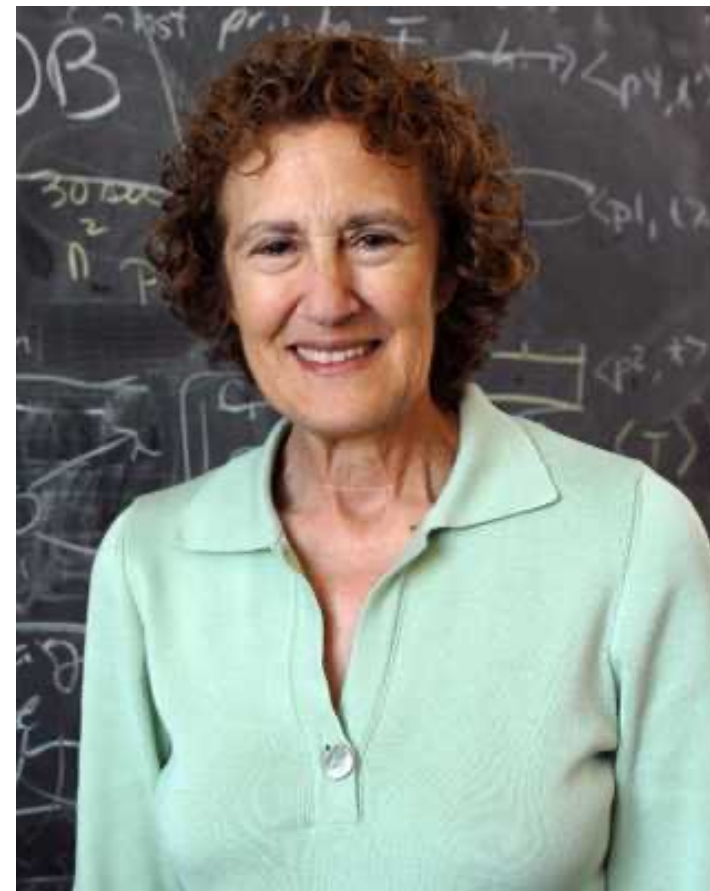
Visibilidade dos membros herdados:

Tipo	Membros public herdados	Membros protected herdados	Membros private herdados
public	public	protected	private
protected	protected	protected	private
private	private	private	private

Princípio da Substituição de Liskov



- O Princípio da Substituição de Liskov define que:
Objetos de uma classe derivada devem poder substituir objetos de sua classe base sem alterar as propriedades desejáveis do programa
- Se B herda de A, então B deve poder ser usado no lugar de A
- Restringir a visibilidade de campos quebra o PSL
 - Seu uso deve ser evitado!



Barbara Liskov (2009)

Construtores e Destrutores



```
class Pessoa {
public:
    Pessoa() {cout << "Contruindo Pessoa\n";}
    ~Pessoa() {cout << "Destruindo Pessoa\n";}
};

class Estudante : public Pessoa {
public:
    Estudante() {cout << "Contruindo Estudante\n";}
    ~Estudante() {cout << "Destruindo Estudante\n";}
};

class EstGrad : public Estudante {
public:
    EstGrad() {cout << "Contruindo EstGrad\n";}
    ~EstGrad() {cout << "Destruindo EstGrad\n";}
};

int main() {
    EstGrad e;
    return 0;
}
```

- Cada classe é responsável por inicializar "sua parte" do objeto
- Ordem de execução:
 - Construtores da classe base
 - Construtores das classes derivadas
- Ordem inversa para destrutores

Construtores e Destrutores



```
class Pessoa {
public:
    Pessoa() {cout << "Contruindo Pessoa\n";}
    ~Pessoa() {cout << "Destruindo Pessoa\n";}
};

class Estudante : public Pessoa {
public:
    Estudante() {cout << "Contruindo Estudante\n";}
    ~Estudante() {cout << "Destruindo Estudante\n";}
};

class EstGrad : public Estudante {
public:
    EstGrad() {cout << "Contruindo EstGrad\n";}
    ~EstGrad() {cout << "Destruindo EstGrad\n";}
};

int main() {
    EstGrad e;
    return 0;
}
```

Qual a saída deste programa?

```
% ./construtores
Contruindo Pessoa
Contruindo Estudante
Contruindo EstGrad
Destruindo EstGrad
Destruindo Estudante
Destruindo Pessoa
```

Construtores e Destrutores



- É possível inicializar os membros da classe base chamando seu construtor explicitamente na classe derivada
 - Necessário se atributo for privado
- Caso não haja chamada do construtor, inicializa a classe base com construtor padrão

```
class Pessoa {  
    string nome;  
public:  
    Pessoa(string _n) : nome(_n) {}  
};  
  
class Estudante : public Pessoa {  
    int matricula;  
public:  
    Estudante(string _n, int _m) : Pessoa(_n), matricula(_m) {}  
};
```



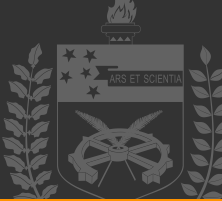
- Técnica para criar um novo tipo pela junção de outras classes de menor complexidade
- Herança $\rightarrow$ é um(a) $\neq$ Associação $\rightarrow$ “*tem um(a)*”
- Não existe uma palavra-chave específica no C++ para isso
- As associações são divididas em duas categorias principais, baseadas no gerenciamento do ciclo de vida dos objetos:
 - Agregação: Relação fraca
 - Composição: Relação forte



- Relação parte-todo com **ciclo de vida independente**
 - A parte pode existir independentemente do todo
- A destruição do objeto principal (todo) não implica na destruição de suas partes (objetos agregados)
- Exemplo: uma Disciplina tem vários Estudantes
 - Se a Disciplina for cancelada ou encerrada, os Estudantes continuam existindo no sistema da universidade
- Geralmente feita via ponteiros para objetos que foram instanciados externamente

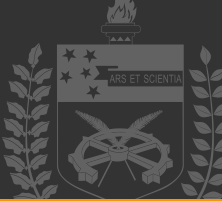


- Relação parte-todo com **ciclo de vida dependente**
 - A parte pertence exclusivamente ao todo
- A destruição do objeto principal (todo) resulta obrigatoriamente na destruição de suas partes (objetos compostos)
- Exemplo: Um Curso é formado por uma grade de Disciplinas
 - Se o Curso for extinto, as suas Disciplinas específicas deixam de existir
- Instanciação interna do objeto (como atributo direto) ou uso de ponteiro com delete explícito no destrutor da classe principal



- Relação mais fraca: "*usa um(a)*"
- Uma classe depende de outra temporariamente para realizar uma operação específica
- A classe utilizada não faz parte do estado do objeto (não é armazenada como atributo)
- O objeto é passado como parâmetro em um método ou instanciado localmente dentro dele

Associações



```
class Estudante : public Pessoa { /* ... */ };

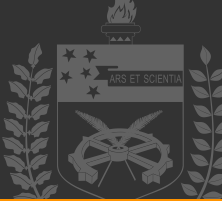
class Disciplina {
    std::string nome;
    std::vector<Estudante*> alunos_matriculados;    // Agregacao

public:
    Disciplina(std::string _n) : nome(_n) {}
    void matricular(Estudante* aluno) {           // Dependencia
        alunos_matriculados.push_back(aluno);
    }
};

class Curso {
    std::string nome;
    std::vector<Disciplina> grade_curricular;    // Composicao

public:
    Curso(std::string _n) : nome(_n) {}
    void adicionarDisciplina(std::string nome_disciplina) {
        grade_curricular.push_back(Disciplina(nome_disciplina));
    }
};
```

Exercício

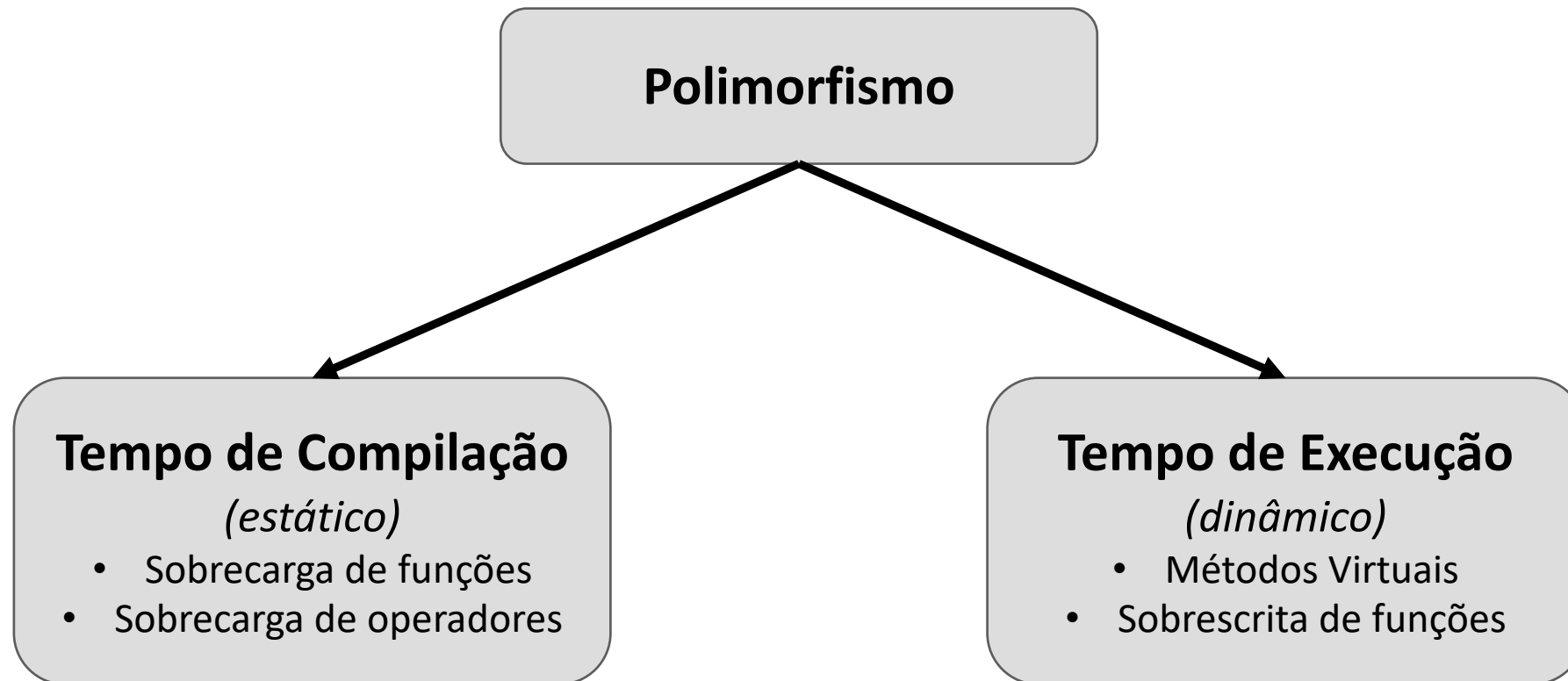


- Crie uma classe base Forma que contenha um identificador único para cada objeto instanciado (um número sequencial, por exemplo)
 - Dica: use uma variável estática para atribuir um valor único ao identificador
- Crie três classes derivadas: Circulo, Retângulo e Triangulo, que herdem da classe Forma
 - As classes derivadas devem estender a classe base para armazenar seus parâmetros geométricos
- Implemente os construtores e o destrutores das classes, de modo que seja possível visualizar mensagens indicando a criação e destruição de cada objeto e seu identificador.



- Polimorfismo → do grego *poli* (muitos) + *morphos* (formas)
- Permite que o mesmo método (ou operador) se comporte de maneiras diferentes em escopos distintos
- Permite uma extensibilidade maior das classes derivadas bem como a programação genérica (mais nas próximas aulas)
- Exemplo: calcular a área de diferentes formas geométricas

- Em C++ temos dois tipos básicos de polimorfismo



Sobrecarga de funções



- Já usamos este tipo de polimorfismo na aula passada...
- Definição de múltiplas funções com mesmo nome
- Assinaturas das implementações são diferentes
 - Retorno e parâmetros

```
class Pessoa {  
    private:  
        std::string nome;  
        float altura;  
  
    public:  
        Pessoa();  
        Pessoa(std::string nome, float _altura, unsigned int _idade);  
        Pessoa(const Pessoa& p);  
};
```

Sobrecarga de funções

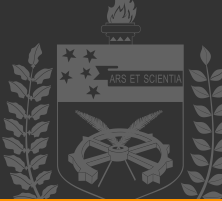


```
class Calculadora {
public:
    int soma(int a, int b) {
        cout << "Some de dois inteiros: "
              << a+b << endl;
        return a+b;
    }
    double soma(double a, double b) {
        cout << "Some de dois doubles: "
              << a+b << endl;
        return a+b;
    }
    int soma(int a, int b, int c) {
        cout << "Some de três inteiros: "
              << a+b+c << endl;
        return a+b+c;
    }
};

int main() {
    Calculadora c;
    c.soma(4, 7);
    c.soma(5.3, 2.2);
    c.soma(1, 2, 3);
    return 0;
}
```

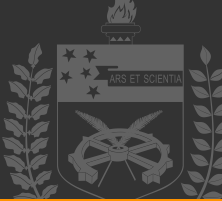
- Escolha da chamada da função é feita pelo compilador
 - Baseado no número e tipo dos parâmetros
- Necessário manter consistência semântica
 - Funções sobrecarregadas executam a mesma coisa
- Pode ser feito fora do escopo de uma classe

Sobrescrita de funções



- Exige uma relação de herança entre duas classes
- Consiste na redefinição do comportamento de uma função da classe base na classe derivada
- Permite o uso de ponteiros para a classe base e resolução dinâmica em tempo de execução do método da classe derivada
- Palavras-chave:
 - `virtual` → declaração de método a ser sobrescrito na classe base
 - `override` → explicitação de método sobrescrito na classe derivada

Métodos virtuais

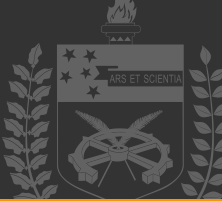


```
class Pessoa {
protected:
    string nome;
public:
    Pessoa(string _n) : nome(_n) {}
    virtual void apresentar() const {
        cout << "Olá, sou " << nome << endl;
    }
};

class Estudante : public Pessoa {
public:
    Estudante(string _n) : Pessoa(_n) {}
    void apresentar() const override {
        cout << "Olá, sou o estudante " << nome << endl;
    }
};

class Professor : public Pessoa {
public:
    Professor(string _n) : Pessoa(_n) {}
    void apresentar() const override {
        cout << "Olá, sou o Prof. " << nome << endl;
    }
};
```

Herança polimórfica



- O uso de classes com métodos virtuais também é chamado de herança polimórfica
- Objeto da classe derivada é um objeto da classe base
 - Definição de polimorfismo e PSL
- Logo pode ser referenciado ou ser apontado por um ponteiro do tipo base

```
int main() {  
    Pessoa *a = new Estudante("Bob");  
    Professor p = Professor("Alice");  
    Pessoa &b = p;  
  
    delete a;  
    return 0;  
}
```

Herança polimórfica



- Uso interessante: vetor de ponteiros da classe base

```
int main() {  
    Pessoa** pessoas = new Pessoa*[3];  
  
    pessoas[0] = new Pessoa("Alice");  
    pessoas[1] = new Estudante("Maria");  
    pessoas[2] = new Professor("Bob");  
  
    for (int i = 0; i < 3; ++i) {  
        pessoas[i]->apresentar();  
        delete pessoas[i];  
    }  
    delete[] pessoas;  
    return 0;  
}
```

```
$ ./polimorfismo  
Olá, sou Alice  
Olá, sou a estudante Maria  
Olá, sou o Prof. Bob
```

- Cuidado!!! Não criamos um destrutor virtual na classe base

Destruidor virtual



```
class Base {
    double *vet_b;
public:
    Base() {vet_b = new double[10];}
    ~Base() {delete[] vet_b;}
    virtual void foo() {}
};

class Derivada : public Base {
    int *vet_d;
public:
    Derivada() {vet_d = new int[10];}
    ~Derivada() {delete[] vet_d;}
    void foo() override {}
};

int main() {
    Base *b = new Derivada;
    delete b;
    return 0;
}
```

==> Valgrind

HEAP SUMMARY:

in use at exit: 40 bytes in 1 block
total heap usage: 4 allocs, 3 frees,

40 bytes in 1 blocks are definitely lost
at 0x484AB64: operator new[](unsigned
by 0x108DC3: Derivada::Derivada()
by 0x108C33: main

LEAK SUMMARY:

definitely lost: 40 bytes in 1 block
indirectly lost: 0 bytes in 0 blocks
possibly lost: 0 bytes in 0 blocks
still reachable: 0 bytes in 0 blocks
suppressed: 0 bytes in 0 blocks

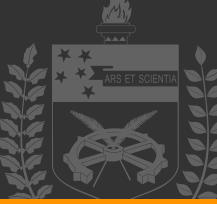
Destruidor virtual



- Em hierarquias polimórficas, a classe base **deve** ter destrutor virtual
- Caso contrario:
 - Destrução incompleta do objeto
 - Chama o destrutor apenas da classe base
- Todas as classes base devem ter um destrutor virtual

```
class Base {  
    double *vet_b;  
public:  
    Base() {vet_b = new double[10];}  
    virtual ~Base() {delete[] vet_b;}  
    virtual void foo() {}  
};
```

Classes abstratas



- Classe que contém pelo menos um método virtual puro

```
virtual tipo nome() = 0
```

- Classes abstratas não podem ser instanciadas
- Define uma interface comum para as classes derivadas
- Muito útil no desenvolvimento de bibliotecas/APIs

Classes abstratas



```
class Forma {
    static unsigned int contFormas;
public:
    Forma() {contFormas++;}
    virtual ~Forma() {contFormas--;}
    virtual double getArea() const = 0;
    static unsigned int getTotalFormas() {return contFormas;}
};

class Circulo : public Forma {
    double raio;
public:
    Circulo(double r) : raio(r) {}
    double getArea() const override {return 3.14*raio*raio;}
};

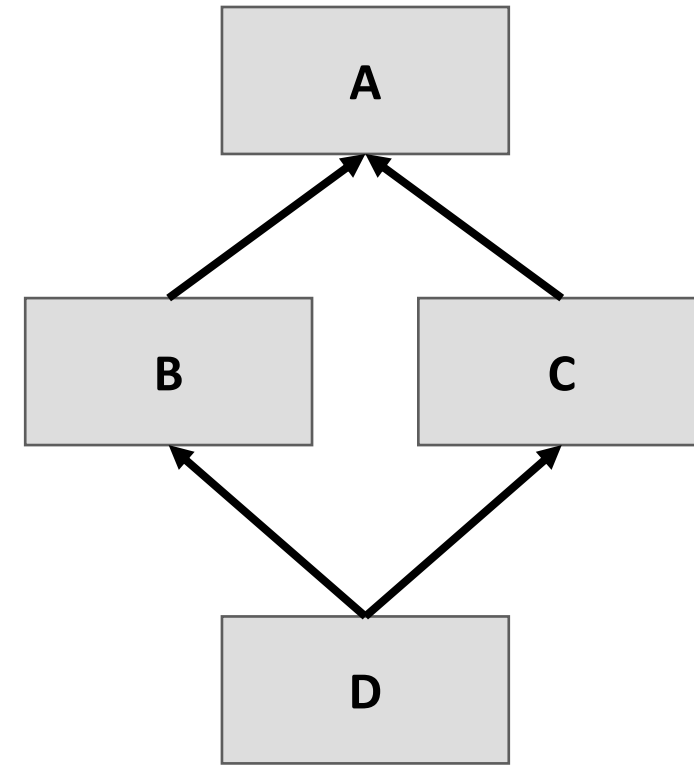
unsigned int Forma::contFormas = 0;

int main() {
    Circulo c = Circulo(2);
    // Forma f; instanciação de classe abstrata não permitida
    std::cout << "Área do círculo " << c.getArea() << std::endl;
    std::cout << "Número de formas " << Forma::getTotalFormas() << std::endl;
}
```


Herança múltipla



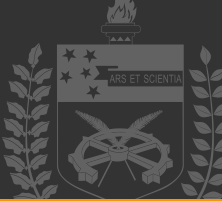
- Herança múltipla
 - Algumas linguagens, como o C++, permitem que uma mesma classe derivada possua diferentes classes base
 - Definições múltiplas do mesmo membro
 - Comportamento não definido, referência definida pelo compilador
 - Problema do diamante
- De maneira geral, a herança múltipla:
 - Dificulta o entendimento e manutenção do código
 - Cria “super-objetos” pouco modulares



Problema do diamante



- Fere o encapsulamento:
 - Membros fazem parte de várias classes
 - Como garantir o principio da substituição?
- Cria interdependência entre as classes
 - Mudanças nas superclasses difíceis
 - Facil de “quebrar” as subclasses
- Se for necessário agrupar comportamentos e características:
 - Melhor solução: Composição



Expanda o exercício anterior para trabalhar com polimorfismo em tempo de execução

- Transforme a classe base Forma em uma classe abstrata, incluindo ao menos um método puramente virtual chamado `getArea()`, que deve ser implementado pelas classes derivadas
- Nas classes derivadas, implemente o método `getArea()` com o cálculo adequado para cada forma
- Crie uma coleção (vetor de ponteiros para a classe base) para armazenar diferentes objetos derivados de Forma
- Demonstre o uso do polimorfismo, percorrendo a coleção e chamando o método `getArea()` de cada objeto, exibindo os resultados